VECTOR DATABASES · NOTEBOOK 02 OF 03

02 — A Knowledge Base for an AI SQL Assistant

A guide to 02_Load_Knowledge_Base_into_Pinecone.ipynb: how a spreadsheet describing a database becomes 337 Pinecone records — and how they turn one business question into a small, correct context for an LLM.

PA

Tutor: Prudhvi Akella
Lead Data & AI Engineer · 13+ years in data engineering and AI architecture

Stack: OpenAI `text-embedding-3-small` · Pinecone serverless · openpyxl (no pandas)
Index: `ecom-kb-tutorial` · Namespace: `ecom-kb` · 337 records from `kb_feed_ecom.xlsx` · about 2½ hours

1. Where 02 sits
2. The end-to-end flow
3. Chapter 1 — The question
4. Chapter 2 — What the LLM doesn't know
5. Chapter 3 — Why not paste everything?
6. Chapter 4 — Six tabs
7. Chapter 5 — The design ★
8. Chapter 6 — Build the records
9. Chapter 7 — Load into Pinecone
10. Chapter 8 — Retrieve
11. Chapter 9 — The context pack ★
12. Chapter 10 — Keep it in sync
13. Chapter 11 — What vector search can't do
14. The embedding model for this knowledge base
15. The code, cell by cell
16. Key takeaways and knowledge check

1 Where 02 sits

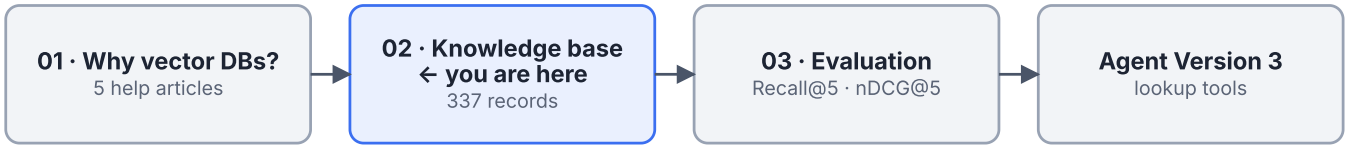


Figure 1. The series. 02 builds the knowledge base the agent searches.

01 stored five help articles to learn the ideas. 02 is the **heart of the course**: the knowledge base a real AI SQL assistant needs. Notebook 03 measures it; Agent Version 3 uses it through its lookup tools.

> **Learning goal:** design a knowledge base — decide for each kind of row what to embed, what id, what metadata and what to skip — and explain how retrieval builds a small, correct context for an LLM.

2 The end-to-end flow

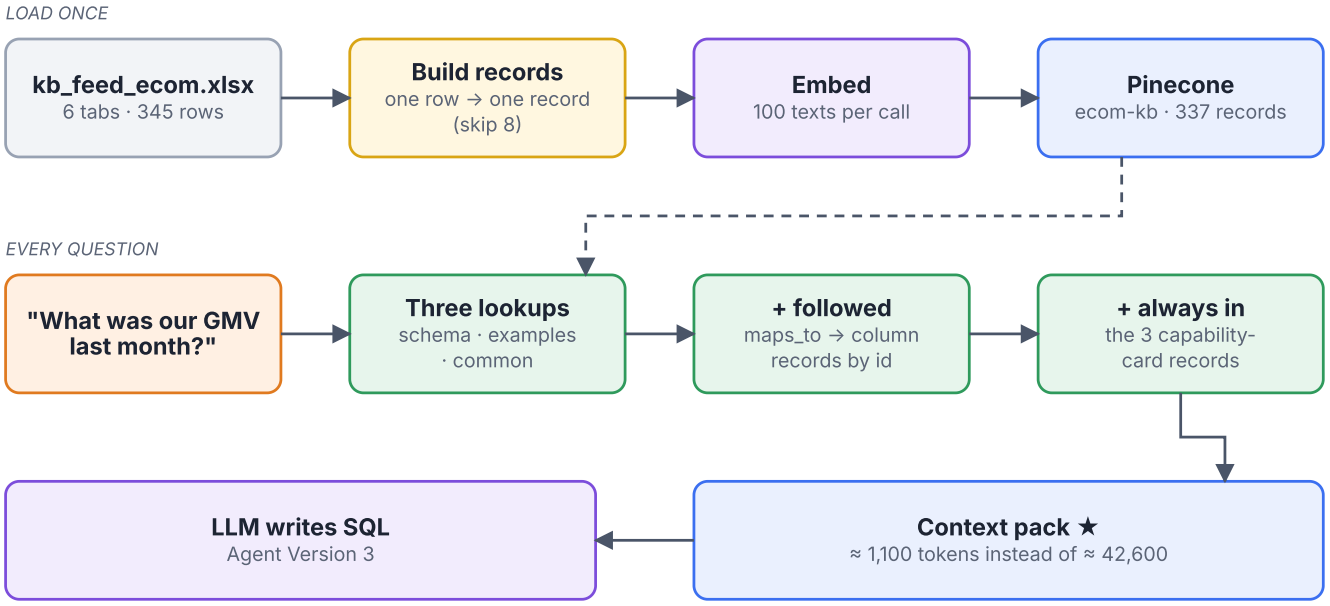


Figure 2. 02 end to end. Top: load the knowledge once. Bottom: build a small context for every question.

Two flows share one Pinecone namespace. **Load once:** read the workbook, build one record per row, embed 100 at a time, upsert. **Every question:** three filtered lookups, plus records followed by id and records always included, assembled into a **context pack** the LLM reads instead of the whole workbook.

1. **Read** `kb_feed_ecom.xlsx` with openpyxl — six tabs, 345 rows.
2. **Build** one record per row with a per-tab builder; skip 8 coordinate rows → **337 records**.
3. **Embed** the record texts, 100 per call, with `text-embedding-3-small`.
4. **Upsert** into the namespace `ecom-kb`, ids stable so reloading replaces rather than duplicates.
5. **Look up** the question three ways: schema, examples, common.
6. **Follow** links in metadata (`maps_to`) and **add** the capability card.
7. **Assemble** the context pack — about 1,100 tokens instead of about 42,600.

Chapter	Topic	Time
1–3	The question · what the LLM doesn't know · why not paste everything	30 min
4–5	Six tabs · the design ★	45 min
6–7	Build · load	20 min
8–9	Retrieve · the context pack ★	35 min
10–11	Sync · limits	20 min

3 Chapter 1 — The question

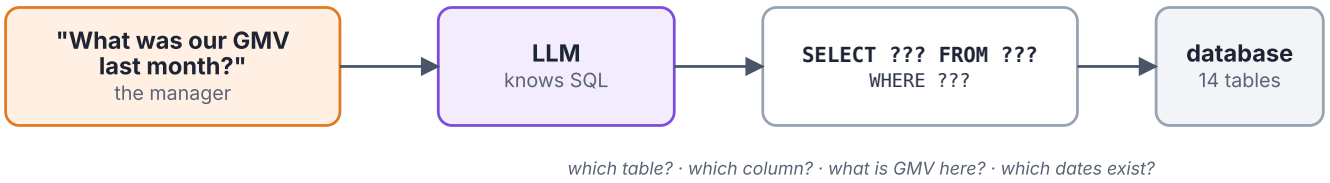


Figure 3. The assistant must turn a business question into SQL — for a database it has never seen.

An online store's database has 14 tables. A manager asks the AI assistant **"What was our GMV last month?"** The assistant is an LLM: it knows SQL in general, but nothing about **this** database. It must decide which table, which column, what GMV means here, and which dates exist — then write the query.

4 Chapter 2 — What the LLM doesn't know

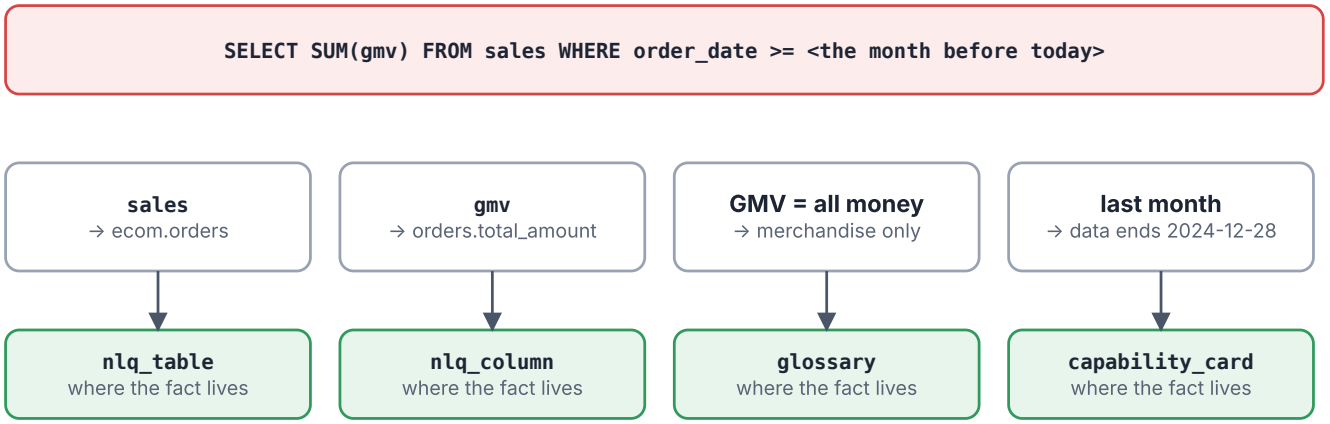


Figure 4. The LLM's likely guess, and where each correction lives in the workbook.

The guess	The truth (from the workbook)	Tab
table `sales`	no such table — orders are in `ecom.orders`	`nlq_table`
column `gmv`	no column is called GMV — it is `orders.total_amount`	`nlq_column`
GMV = everything paid	here GMV = **Revenue** = merchandise only; shipping, tax and discount not included	`glossary`
"last month" = before today	orders run 2023-01-01 to **2024-12-28**; nothing later	`capability_card`

Two more facts make the answer *good*: a **proven query** for revenue by month (`nlq_examples`), and **how tables connect** when a question needs several (`nlq_join`).

> **Next question:** the LLM needs these facts with every question. **Why not simply give it all of them, every time?**

5 Chapter 3 — Why not paste everything?

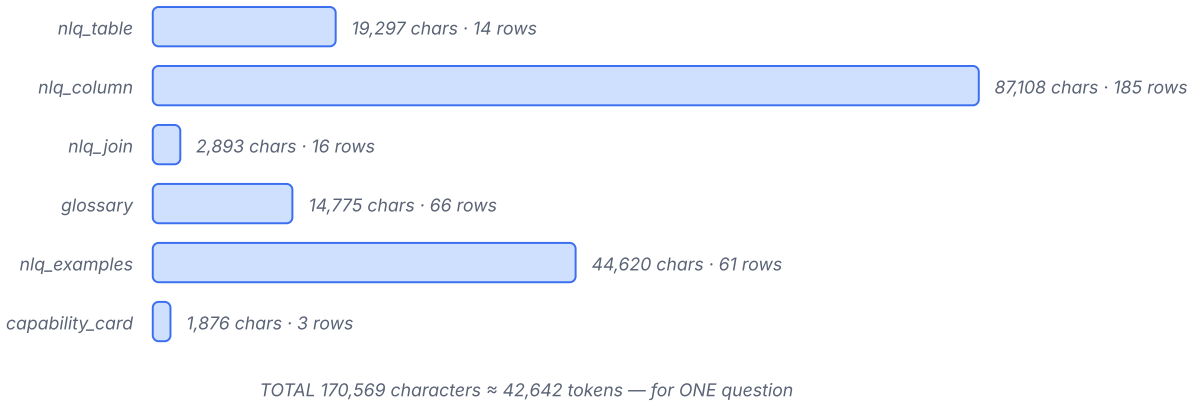


Figure 5. The whole knowledge, measured: ≈ 170,600 characters ≈ 42,600 tokens per question if pasted in.

The notebook measures it: **≈ 170,600 characters ≈ 42,600 tokens** for a 14-table database.

Problem	Why it hurts
Cost and speed	every question pays for ~42,600 input tokens, almost all irrelevant to it
Noise	the LLM must pick 5 useful facts out of 345 rows — more chances to choose a wrong column
Scale	a real company has hundreds or thousands of tables; it won't fit at all

The fix — RAG (Retrieval-Augmented Generation): store the knowledge in a vector database; for each question **retrieve only the pieces that matter**, and give the LLM a small, focused context.

Next question: what knowledge do we store, and **how**?

6 Chapter 4 — Six tabs

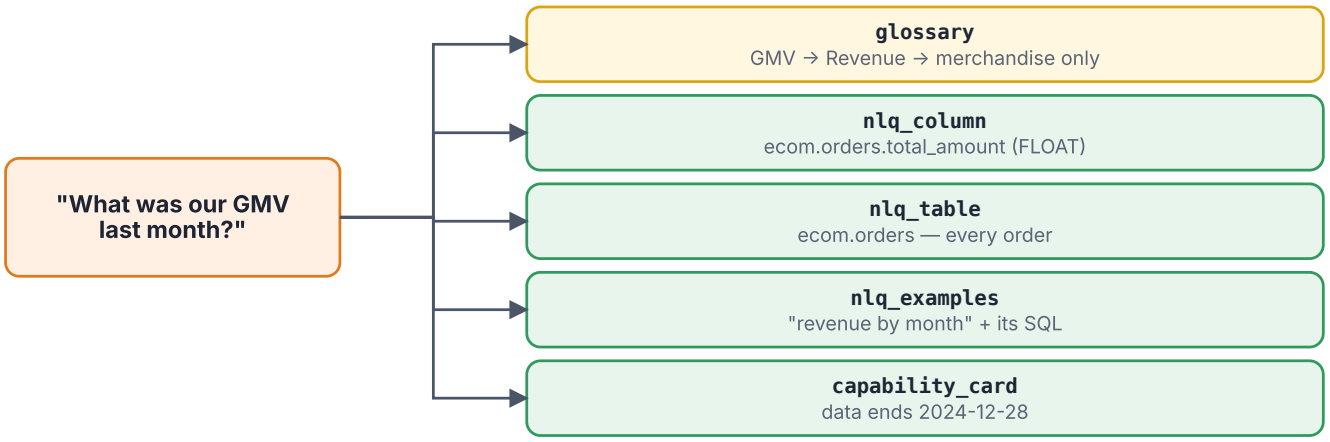


Figure 6. One question needs facts from five of the six tabs.

Tab	Rows	Answers the LLM's question	Example row
`nlq_table`	14	what tables exist, what is in each?	`ecom.orders` — every order
`nlq_column`	185	which column holds this? what do people call it?	`orders.total_amount` — also called Revenue
`nlq_join`	16	how do two tables connect?	orders → customers on `customer_id`
`glossary`	66	what does this business word mean here?	Revenue — also called GMV
`nlq_examples`	61	is there a proven query to copy?	"Which brands generate the most revenue?" + SQL
`capability_card`	3	what can and can't be answered?	orders after 2024-12-28 can't

The `Instructions` tab is for the people who maintain the workbook, not for the assistant — it isn't loaded.

> **Next question:** each row becomes one record. **What exactly goes into each record?**

7 Chapter 5 — The design ★

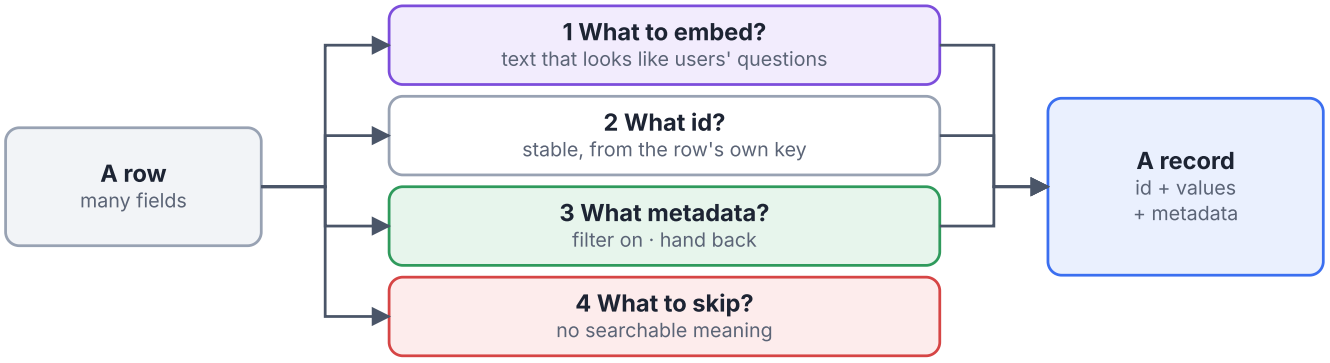


Figure 7. Four decisions turn a spreadsheet row into a Pinecone record.

A Pinecone record is **id + values (the embedding) + metadata** (from 01). For every tab we make four decisions:

Decision	Question to ask	Rule
1 What to embed	what will users' questions look like?	embed text that resembles the question — descriptions, synonyms, example questions
2 What id	how do I find this record again to update or delete it?	a stable name from the row's own key — never a random number
3 What metadata	what will I filter on? what must come back with a match?	labels to filter (`sheet` , `table`) + what the LLM needs (text, SQL)
4 What to skip	does this row or field carry meaning a search can use?	skip what has no searchable meaning

The six walk-throughs below follow the GMV story: the table, the column, how it connects, what GMV means, a proven query, and the limits. Each shows the real row on the left and the record the code builds on the right.

1 · nlq_table — what tables exist

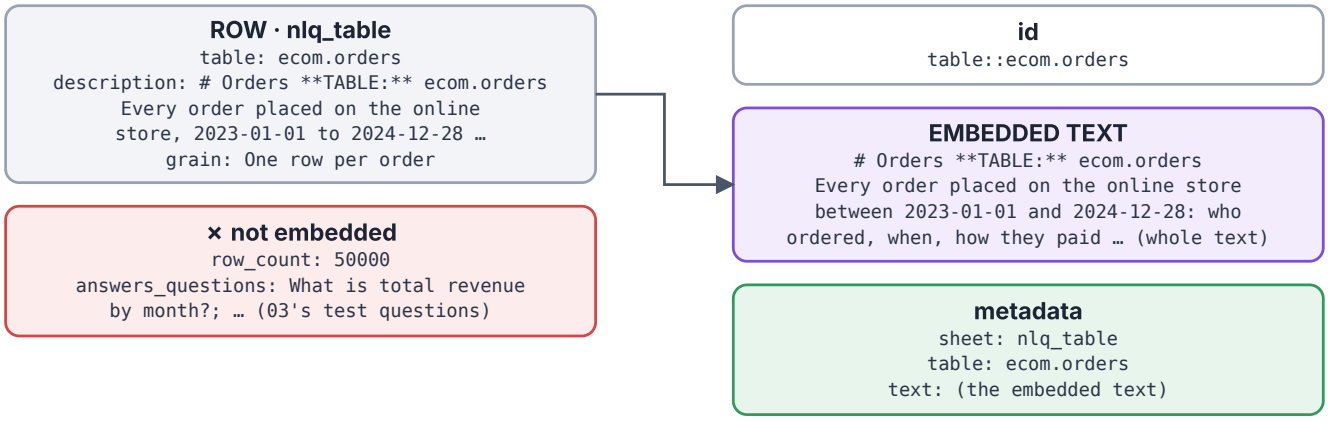


Figure 8. A table row. The Markdown description is embedded whole; row_count and answers_questions are not.

The description already is a full write-up — grain, measures, meaning — so it's embedded as it is. `answers_questions` stays out **on purpose**: notebook 03 uses those questions as its test. Embed them and the test becomes an open-book exam.

2 · nlq_column — which column holds it

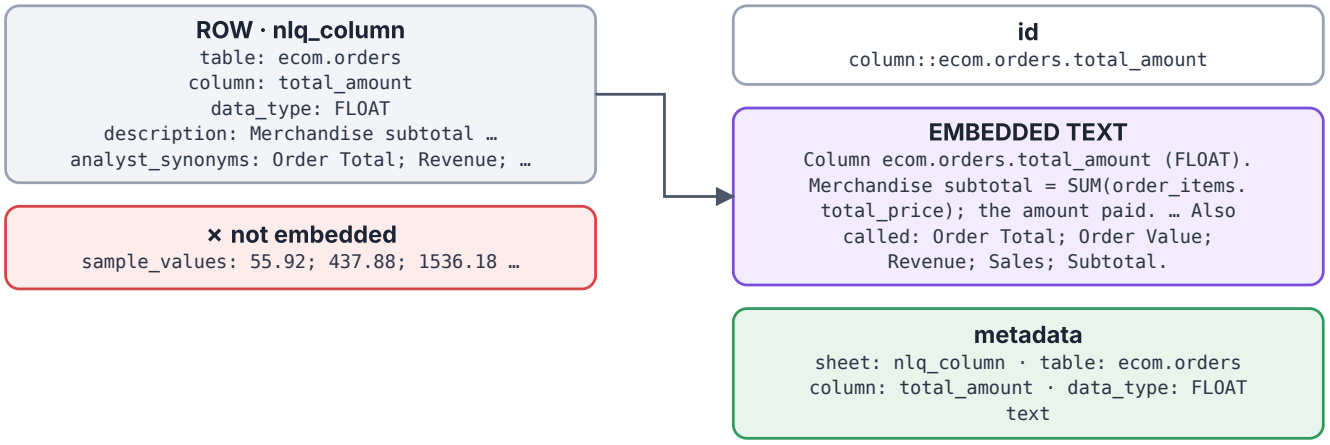


Figure 9. A column row. Synonyms go into the text — users say "revenue", not "total_amount".

Sample values are numbers — "55.92" means nothing to a meaning search. `table` and `column` are metadata so a search can be narrowed to one table. **8 column rows** (map coordinates such as `lat`, `lon`) are skipped entirely: `embed_exempt = Y`.

3 · nlq_join — how two tables connect

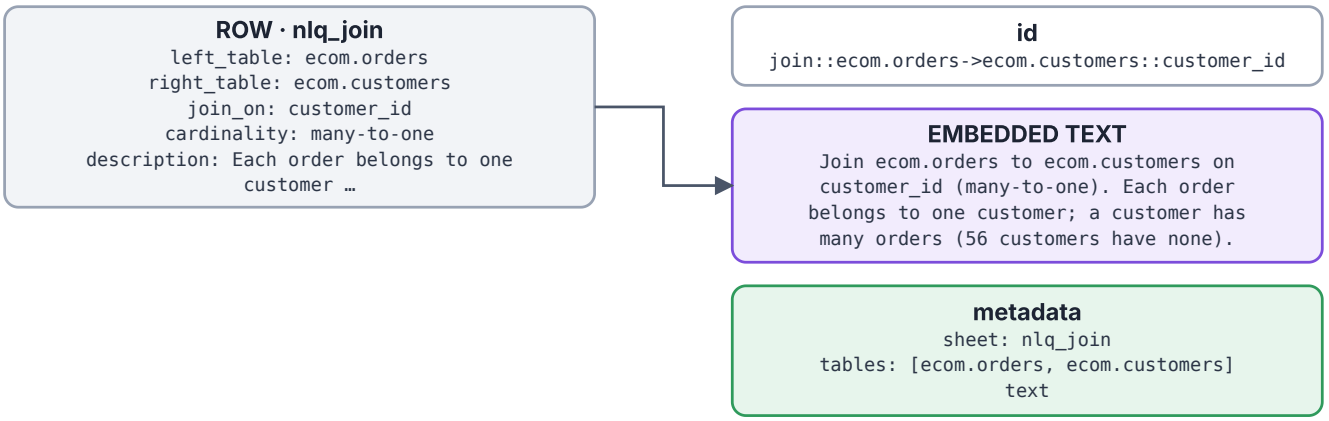


Figure 10. A join row. Five short fields are written as one sentence that reads like the question it answers.

Both tables go into one metadata **list**, `tables` — a filter on either table finds the join.

4 · glossary — what a business word means here

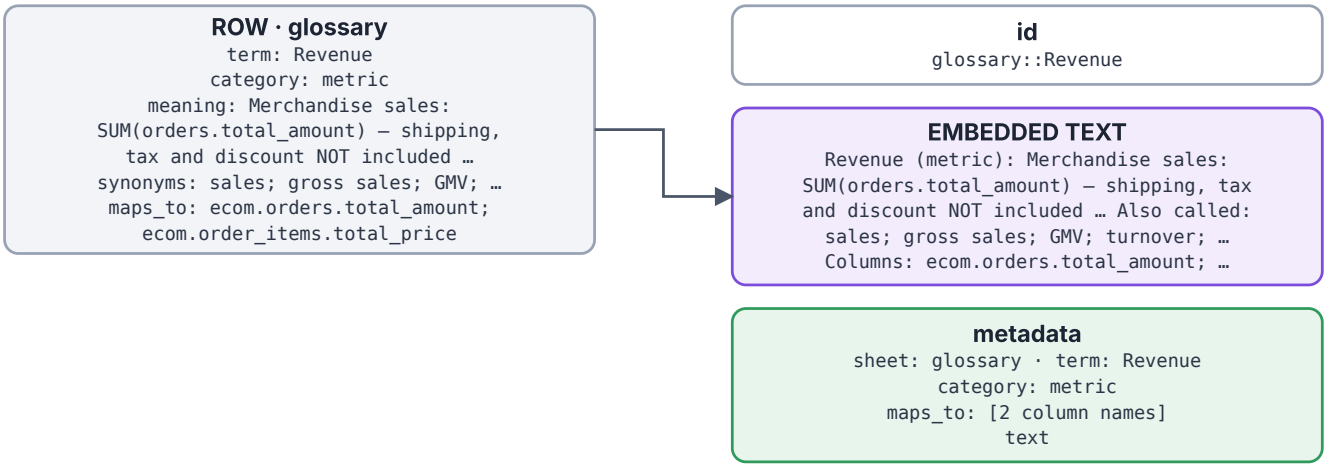


Figure 11. A glossary row. The synonyms are embedded — that is how "GMV" reaches Revenue.

> **GMV appears in exactly one place in the whole workbook: this row's synonyms.** Embed them, and "What was our GMV…" lands here. Leave them out, and the model must guess. This is how we teach the assistant **company jargon**. `maps_to` is a metadata list that chapter 9 follows to fetch the exact columns.

5 · nlq_examples — a proven query to copy

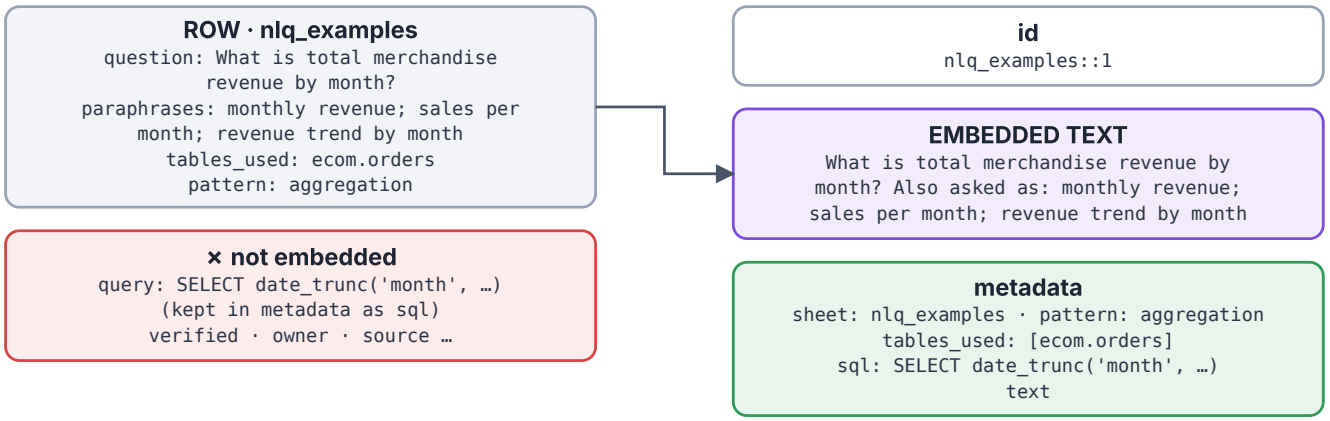


Figure 12. An example row. The question is embedded; the SQL rides along in metadata.

Users ask questions, not SQL — so the **question and paraphrases** are embedded and the **SQL** comes back with the match, ready for the LLM. Bookkeeping fields help the people maintaining the workbook, not the search.

6 · capability_card — what can't be answered

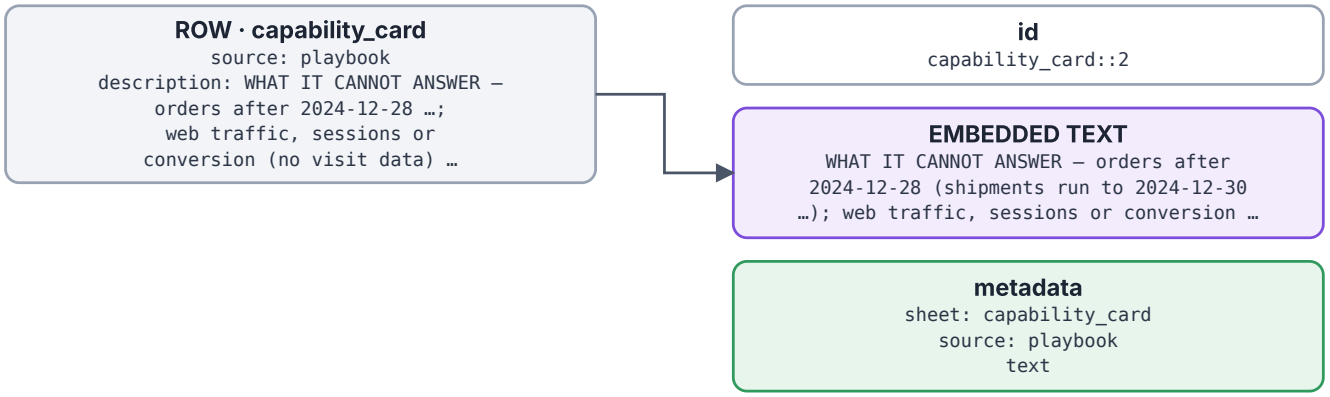


Figure 13. A capability-card row. Embedded as it is — and, in chapter 9, always included.

Embedded so "website visitors per day" can find "no visit data". Only 3 such records exist and every question needs the limits, so chapter 9 also **always includes** them.

The design for all six tabs

Tab	Embed	Id	Metadata	Skip
`nlq_table`	the description	`table::<table>`	sheet, table · text	row_count, answers_questions
`nlq_column`	name, type, description, synonyms, values	`column::<table>.<column>`	sheet, table, column, data_type · text	8 rows (`embed_exempt`); sample values
`nlq_join`	one sentence from five fields	`join::<left>-><right>::<key>`	sheet, tables[ ] · text	—

Tab	Embed	Id	Metadata	Skip
`glossary`	term, meaning, **synonyms**, columns	`glossary::<term>`	sheet, term, category, maps_to[ ] · text	—
`nlq_examples`	**question** + paraphrases	`nlq_examples::<n>`	sheet, tables_used[ ], pattern · text, **sql**	the SQL (kept as metadata)
`capability_card`	the description	`capability_card::<n>`	sheet, source · text	—

Pinecone rules the code respects: metadata values must be text, numbers, true/false or lists of text — **never empty** — and under 40 KB per record (our largest is about 2.6 KB). Stable ids matter because the workbook changes (chapter 10). `nlq_examples::<n>` uses the row number — fine here; in production, build ids from a key column so reordering rows can't change them.

8 Chapter 6 — Build the records

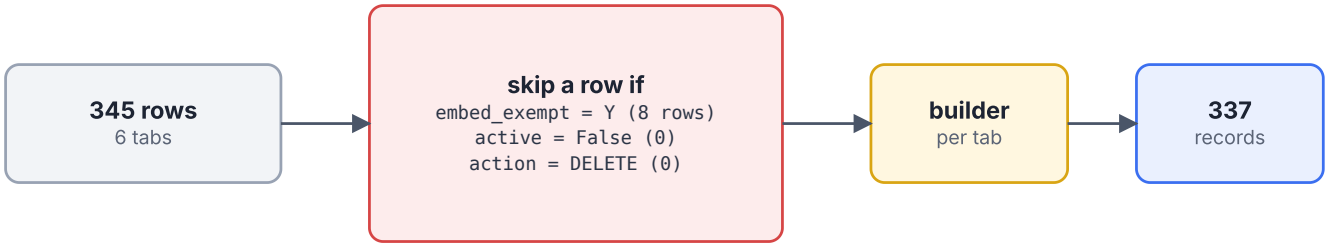


Figure 14. Three skip rules, then one builder per tab: 345 rows → 337 records.

Tab	Rows	Kept	Skipped
`nlq_table`	14	14	0
`nlq_column`	185	177	8 (`embed_exempt = Y`)
`nlq_join`	16	16	0
`glossary`	66	66	0
`nlq_examples`	61	61	0
`capability_card`	3	3	0
Total	**345**	**337**	**8**

All three skip rules are checked even though `active` and `DELETE` match nothing today — the workbook will change. The cell ends by checking that **no two records share an id**.

9 Chapter 7 — Load into Pinecone



Figure 15. 100 records per call: 4 embedding calls instead of 337.

Each call embeds 100 texts and upserts 100 records into `ecom-kb` — the same index as 01, a new namespace, so `help-articles` and `ecom-kb` never mix. Re-running the cell replaces the same ids: no duplicates. The cell then waits until all 337 records are searchable.

10 Chapter 8 — Retrieve

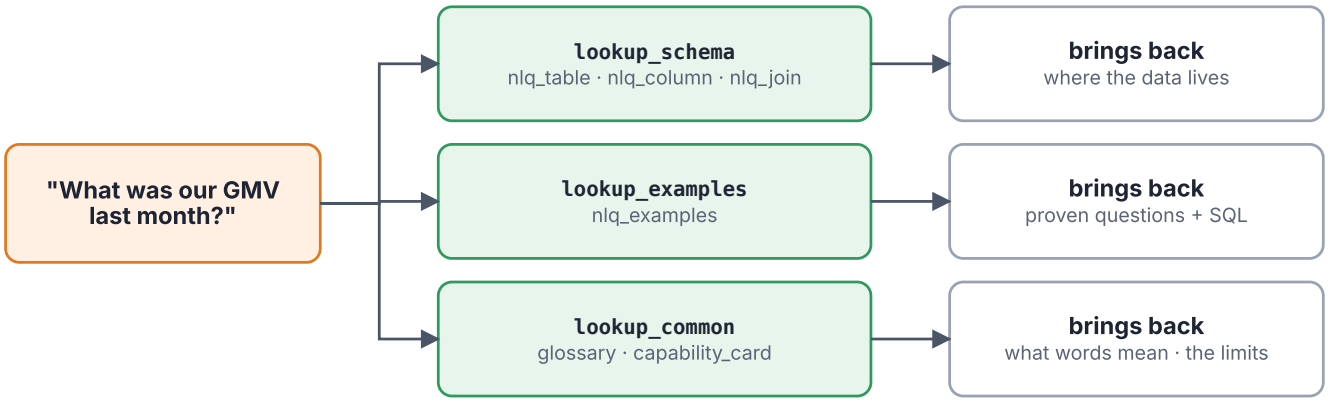


Figure 16. The agent's three lookups: the same question, searched in different tabs with a metadata filter.

First the notebook searches the **whole** knowledge base with the GMV question. The result is a mix: `glossary::Revenue` (reachable only through "GMV"), revenue examples — and some noise, such as columns with "last" in their name for "last month". Everything competes for the same top-K slots.

So the agent runs **three lookups**, each limited by the `sheet` metadata — the filter from 01:

```
lookup(question, sheets=["glossary", "capability_card"], top_k=3)
# → index.query(..., filter={"sheet": {"$in": ["glossary", "capability_card"]}})
```

Agent tool (Version 3)	Searches	Brings back
`lookup_schema`	`nlq_table`, `nlq_column`, `nlq_join`	where the data lives
`lookup_examples`	`nlq_examples`	proven questions + their SQL
`lookup_common`	`glossary`, `capability_card`	what words mean here, and the limits

11 Chapter 9 — The context pack ★

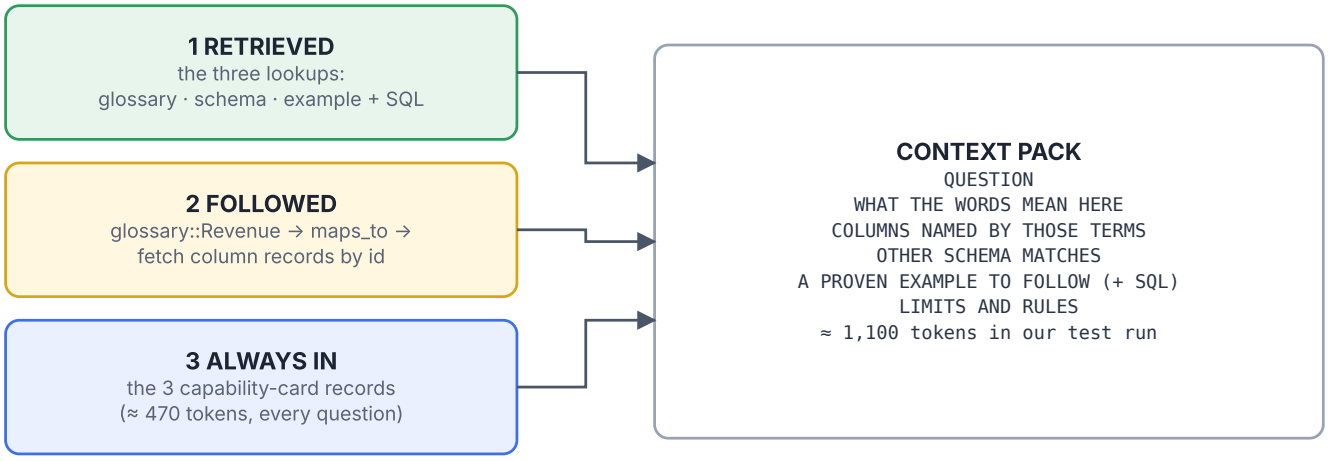


Figure 17. Three routes into the context pack. The pack is what the LLM reads instead of the workbook.

This is the point of the notebook. A fact gets into the pack one of three ways:

Route	How	Why
1 Retrieved	the three lookups	the search finds what is similar to the question
2 Followed	`glossary::Revenue` → its `maps_to` → `index.fetch(ids=["column::ecom.orders.total_amount", …])`	a search can rank the right column low; a link in metadata reaches it **by id** — no guessing
3 Always in	fetch `capability_card::1–3` by id	small, always-relevant knowledge isn't left to a search

Chapter 2 needed	In the pack as	Route
what GMV means here	Revenue = merchandise only	retrieved
the column and table	`ecom.orders.total_amount`	**followed**
a proven query	"What is total merchandise revenue by month?" + SQL	retrieved
whether "last month" has data	orders end 2024-12-28	**always in**

Retrieve what is large; follow links you trust; always include what is small and always needed. In our test run the pack was about **1,100 tokens** — against about 42,600 for the whole workbook. If the glossary term itself is missing, it's a retrieval problem: fix the knowledge text or the lookups, not the LLM. Notebook 03 measures how often retrieval gets it right.

12 Chapter 10 — Keep it in sync

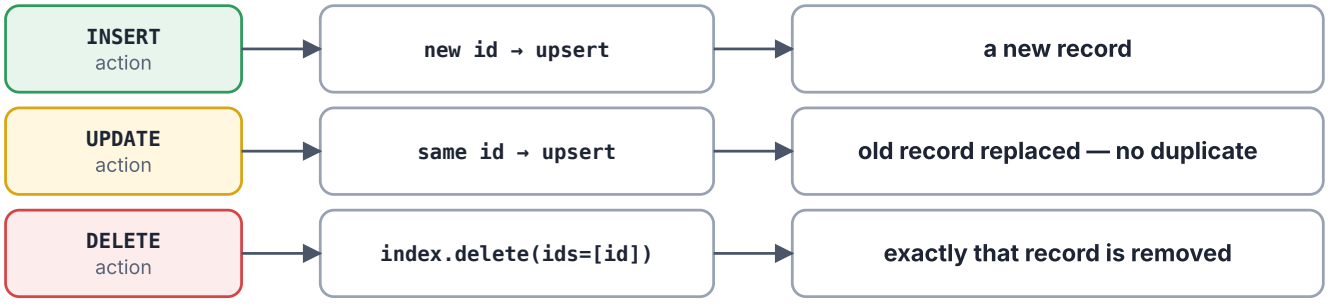


Figure 18. The workbook's action column, and what it means in Pinecone. Stable ids make it work.

The database changes, so the workbook changes, so the index must follow. The notebook demonstrates it on a throw-away record — `glossary::Test Term`: insert, read back by id, upsert again with the same id (replaced, not duplicated), delete. None of the 337 real records is touched.

13 Chapter 11 — What vector search can't do



Figure 19. "Revenue by supplier" needs a path through five tables. Vector search returns pieces, not paths.

Ask the schema lookup **"What is revenue by supplier?"** It returns supplier columns, the supplier table and a join or two — records **similar in meaning**. It does not work out a **path**: which joins, in which order. That is structure, not meaning. The agent follows the `nlq_join` records **in code**; vector search finds the pieces, code connects them.

14 The embedding model for this knowledge base

01 explains how to choose a model (five questions → shortlist → test). Applied to this knowledge base:

Question	This knowledge base	Consequence
May the text leave the company?	yes — sample data	an API model is fine
Languages	the workbook is English; questions may not be	test Telugu/Hinglish questions in 03
Record length	longest texts are table descriptions — under 2,000 characters	far below 8,191 tokens; no chunking needed

Question	This knowledge base	Consequence
Cost and size	337 records ≈ 42,600 tokens to embed	well under a cent; 337 × 6 KB ≈ 2 MB of vectors
Which finds the right records?	measured in notebook 03	Recall@5 · nDCG@5 per tab

Changing the model later (say, to Amazon Titan Text Embeddings V2 in Bedrock, 1024 dimensions): change `EMBED_MODEL` and `embed()`, create a **new index** with dimension 1024, reload all 337 records, and embed every question with the new model. Then run 03 on both and compare. Same model for records and questions — always.

15 The code, cell by cell

Cell	What it does	Does NOT
setup	keys, `embed()`, connect to `ecom-kb-tutorial`	load data
measure	`read_sheet()`; rows and characters per tab; total ≈ tokens	embed anything
builders	one function per tab → (id, text, metadata); `clean()` drops empty values; prints the six walk-throughs	embed or upload
build all	skip rules → 337 records; checks ids are unique	embed
load	embed + upsert 100 at a time; wait for 337	delete anything
lookup	`lookup(question, sheets, top_k)` + whole-KB search	print the SQL
three lookups	schema · examples · common for the GMV question	decide which facts to use
context pack	retrieved + followed + always in → one text; size vs the workbook	call an LLM
more questions	the pack for three other questions	—
sync	insert → update → delete a test record	touch the 337 real records
limits	schema lookup for "revenue by supplier"	find a join path

16 Key takeaways and knowledge check

The story in one breath: an LLM can write SQL but doesn't know this database → the facts it needs are in the workbook, but all of it is ~42,600 tokens per question → so each row becomes a **record** (embed what questions look like, a stable id, metadata to filter, link and hand back) → 337 records go into Pinecone → for each question the agent runs **three lookups**, follows the links, adds the limits → and sends a small **context pack** → stable ids keep the index in sync → and code, not vector search, connects tables.

#	Question	Answer
1	Two facts the LLM can't know for the GMV question?	any two: `ecom.orders` · `total_amount` · GMV = merchandise-only Revenue · data ends 2024-12-28
2	Why not paste the whole workbook?	~42,600 tokens per question: costly, slow, noisy — and it doesn't scale
3	The four design decisions?	what to embed · what id · what metadata · what to skip
4	Why embed the glossary synonyms?	"GMV" appears only there; embedding it lets GMV questions find Revenue
5	Why is the SQL metadata, not embedded?	users ask questions, not SQL; the SQL comes back with the match
6	Why stable ids?	an UPDATE replaces the same record; a DELETE knows exactly which
7	Which rows are skipped?	`embed_exempt = Y` (8), `active = False`, `action = DELETE`
8	The three lookups?	`lookup_schema` · `lookup_examples` · `lookup_common`
9	Three routes into the context pack?	retrieved · followed (maps_to, by id) · always in (capability card)
10	Why can't vector search answer "revenue by supplier" alone?	it finds similar pieces, not a join path; code follows `nlq_join`

Next — 03: measure how often retrieval finds the right records, with ground truth.